

Neural Networks from Scratch

Technical Report – Type A Project

Andres Lazzari

Dipartimento di Informatica
Università di Pisa

Leonardo Elmi

Dipartimento di Fisica
Università di Pisa

January 23, 2026

Abstract

This report details the implementation of a Multilayer Perceptron (MLP) neural network with the SGD learning algorithm from scratch in Python. The performance of the simulator was evaluated using the MONK datasets (1, 2, and 3) as well as the CUP dataset provided to us by Prof. Micheli. Key techniques investigated include optimization with momentum, L2 regularization as well as a comparison of the three main learning algorithms: online, batch and minibatch learning.

“In the long run, curiosity-driven research just works better...
Real breakthroughs come from people focusing on what they're excited about.”

— *Geoffrey Hinton*

Introduction and objectives

Methods

MONK Results & Discussion

Conclusions

Introduction and objectives

Introduction

The aim of this project is to develop and evaluate a multilayer perceptron neural network implemented from scratch in and apply it to classification tasks and to a regression task on the CUP dataset. Objectives include:

- Compare learning strategies and their capability to generalise.
- Track performance metrics: MSE, MEE and accuracy for different sets of hyperparameters.
- Gain first-hand experience on the direct application of theoretical principles to real systems.

Methods

Methods: Overview

- Modular architecture for both classification and regression tasks.
- Responsibility division between single scripts with abstract classes, subclasses and their methods.
- Key scripts: `run_monk.py` and `run_cup.py`.

Config-Driven Execution

- Hyperparameter configuration files:
 - `monk_config.py` settings for MONK tasks
 - `cup_config.py` settings for CUP tasks
- Allows a very easy configuration of the grid search over hyperparameter space.
- No changes to core code needed; just modify configs.

Architecture & Optimization

- **Activation:** Configurable hidden and output layers with Sigmoid, ReLU, Leaky ReLU and Tanh.
- **Learning:** Capability to switch among online, batch, minibatch strategies, implementation of a rudimental learning rate decay (proportional to epochs) and Momentum (α_{mom}) to add inertia to weight updates.
- **Regularization:** L2 regularization factor (`lambda12`) and Early Stopping with ϵ and patience thresholds.
- **Weight initialisation:** We used uniform He initialisation for ReLU and Leaky ReLU activation functions and uniform Glorot for Sigmoid and tanh activation functions

Model Selection strategy

- **MONK:** Stratified hold-out (80% train / 20% validation) to have a more significant validation measure for the grid search, final retrain with the best configuration on TR+VL with 20 different weights initialisations (only for monk 3) for model assessment.
- **CUP:** Simple hold out to keep 20% for the internal TS → Grid search with 3-fold CV on each configuration, followed by a retrain with the number of epochs found during grid search on TR+VL sets with the best model for model assessment.

MONK Results & Discussion

MONK Results — Summary

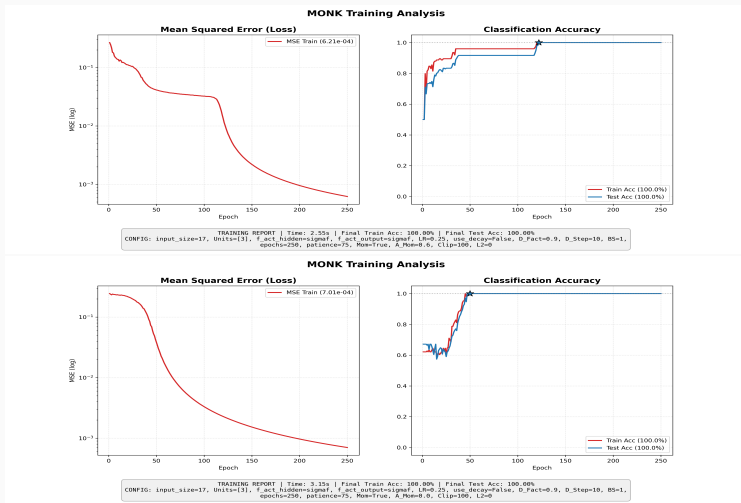
Table 1: Best configurations found (MONK 3 results averaged over 20 runs with different initialisations, error calculated with standard deviation)

Task	Hyperparameters (Units, η , batch, α_{mom})	MSE (Train/Test)	Acc. (Tr / Te) %
MONK 1	3, 0.25, 1, 0.6	$6.21 \cdot 10^{-4}$ / $6.26 \cdot 10^{-4}$	100 / 100
MONK 2	3, 0.25, 1, 0.0	$7.01 \cdot 10^{-4}$ / $7.33 \cdot 10^{-4}$	100 / 100
MONK 3 (Reg 10^{-2})	3, 0.25, 98, 0.9	$(6.5 \pm 0.2) \cdot 10^{-2}$ / $(4.9 \pm 0.1) \cdot 10^{-2}$	93.44 ± 0.01 / 97.21 ± 0.05
MONK 3 (No Reg/ES)	3, 0.25, 98, 0.9	$(4.8 \pm 0.1) \cdot 10^{-2}$ / $(6.5 \pm 0.2) \cdot 10^{-2}$	94.71 ± 0.48 / 95.83 ± 0.30

Table 2: Search Space Explored (864 to 1728 configs)

Task	Units	Act.	η	Pat.	α_{mom}
MONK 1/2	[2]→[3,3]	Sig., ReLU	0.001→0.25	—	0→0.9
MONK 3	[2]→[3,3]	Sig., ReLU	0.001→0.25	5→50	0→0.9

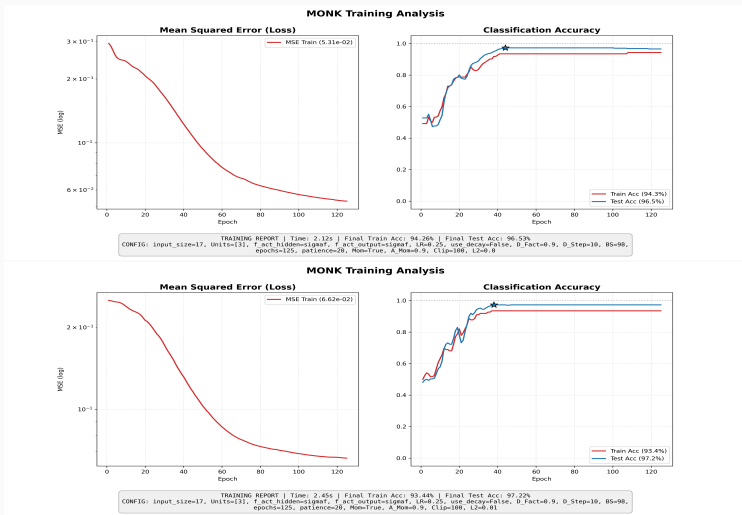
MONK 1 & 2: Analysis



- Both tasks achieved **100% Accuracy** and negligible MSE ($< 10^{-3}$).
- **Topology:** A small architecture (3 hidden units) was sufficient to learn the logical rules.
- **Strategy:** Online learning (Batch=1) proved most effective for these tasks.
- **Momentum:** MONK 1 benefited from moderate momentum ($\alpha = 0.6$), while MONK 2 converged best without it ($\alpha = 0.0$).
- **Note:** Differentiating best models was difficult as multiple configurations reached 100% accuracy.

Figure 1: Monk 1 and 2 final results, test and validation

MONK 3: Handling Noise



Statistical Assessment (20 runs)

plots shown are sample plots taken among the 20 runs for each the unregularised (top) and regularised (bottom) model

- **Challenge:** MONK 3 contains noise. Minimizing training error too far degrades generalization.
- **No Reg/ES:** Disabling Early Stopping and L2 reg. allowed the model to overfit ($MSE \approx 0.048$, Tr Acc 94.7%), dropping Test Acc to 95.8%.
- **With L2:** Regularization accepted a higher training error ($MSE \approx 0.065$) but yielded the optimal Test Acc ($97.21\% \pm 0.05\%$).

Figure 2: Monk 3 final results with and without regularisation

MONK Discussion: Learning Dynamics

Impact of Batch Size

- *Online (Batch=1)*: Superior for deterministic tasks (MONK 1/2). Stochasticity aided in escaping local minima.
- *Full batch*: performs well for MONK 3. Averaging the gradient over the whole dataset minimized the impact of individual noisy outliers.
- *minibatch* didn't come out as a winner in our trials, but performed well on all tasks

Bias-Variance Tradeoff

- *Overfitting Demo*: By removing Early Stopping and L2, we observed the classic overfitting gap: Training Accuracy increased significantly (+1.3%), but Test Accuracy degraded (−1.4%).
- *Solution*: The combination of **L2 Regularization** ($\lambda = 0.01$) and **Early Stopping** effectively bounded the model complexity, preventing it from memorizing the noise.

Table 3: Best configuration performance

Metric	Hyperparameters ($[500, 250]$, $\eta = 0.02$, $batchsize=142$, $\alpha_{mom} = 0.9$, $\lambda_{L2} = 10^{-6}$)	Training/Validation (GS)	Training 2 (on TR+VL)	Internal Test	Training 3 (TR+VL+TS)
MSE	//	$489.3 \pm 59.6 / 621.7 \pm 34.3$	347.6	580.0	110.0
MEE	//	$19.68 \pm 1.13 / 22.44 \pm 0.56$	15.9	20.8	9.2

Table 4: Hyperparameter Search Range (Multiple grid search runs)

Param	Architecture	Activation	Optim.	Reg.
Range	Layers: 2 to 4 Units: 32 $\rightarrow$ 800 Batch: 32 $\rightarrow$ Full	Hidden: Leaky ReLU Output: Linear	η : $10^{-4} \rightarrow 0.02$ α : 0.0 $\rightarrow$ 0.9	L2: $10^{-6} \rightarrow 0$ ϵ : 10^{-11} to 10^{-6} Pat: 15 $\rightarrow$ 200

CUP results, grid search discussion

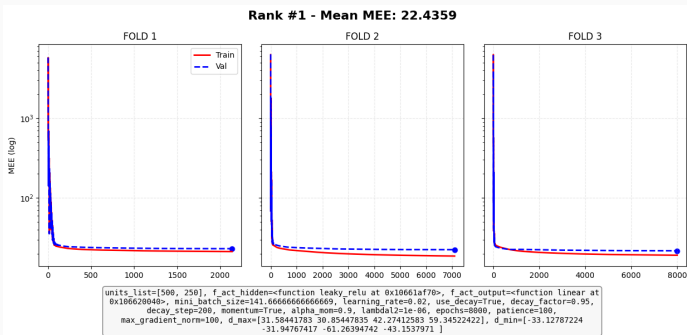


Figure 3: best models during grid search

Approach

The only preprocessing that was done on the cup data is a simple normalisation to the range [0,1].

A preliminary analysis of the dataset with trial runs revealed that low complexity architectures never performed well, therefore our study is focused around relatively high-complexity models, the CUP dataset is a much more complex and noisy dataset we soon shifted to a "pruning like" approach, made many preliminary tests to understand the range of parameters to focus on while comparing the models we found using the VL set.

From our understanding two layers performed the best with a number of units ranging from 100 to 600, here online learning was not tested thoroughly since we very soon discovered that the learning curve would come out rapidly oscillating, full batch and half batch (batch size = 50% of TR patterns) were our preferred choices, we ran the best models for up to 8000 epochs using our η decay mechanism which seemed to work sufficiently well to avoid oscillating around a minimum, although high epoch runs were limited due to the very high computing time needed.

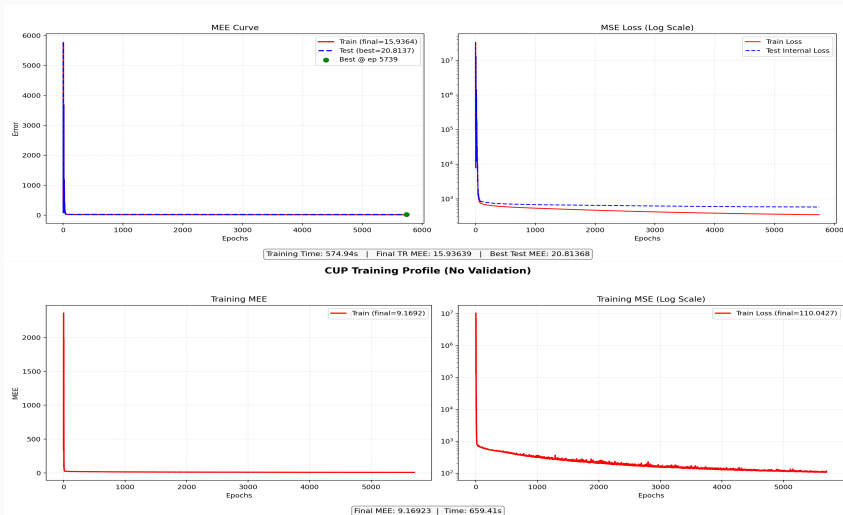


Figure 4: Training 2 (on TR+VL) learning curve with internal test set and Training 3 learning curve

CUP Discussion

- **Learning Curves:** Learning curves are smooth mostly thanks to the usage of full batch learning.
- **Model Selection:** During the preliminary runs we tried to overfit the data but probably due to the complexity of the dataset was very hard. Even with small L2 regularisation and small ϵ , our model's capabilities to generalise did not decrease even with a relatively high amount of training epochs.
- **Retraining Strategy:** For the Grid Search, we utilized Early Stopping on each of the 3 folds. To produce the final model, the retraining on the combined set (TR+VL) was performed for a fixed number of epochs, calculated as the **average** of the stopping epochs observed across the three folds.
- **Blind Test:** Finally the best model, after being retrained on TR+VL, was assessed with the internal TS, only afterwards it was retrained for a second time over the whole TR+VL+TS(internal) the resulting model is our final choice for the CUP.

The hardware at our disposal are: Intel Core Ultra 5 125H, Apple Silicon M2, AMD Ryzen 5 5600. Execution time is around 200s for each fold of each configuration during the grid search (varies heavily with the configuration being tested) training time for the final model 659s.

Conclusions

Conclusions

- **Implementation Success:** We developed a modular MLP library from scratch in Python, validating its correctness on both logical classification (MONK) and complex regression (CUP).
- **MONK Tasks (Classification):**
 - **Simplicity Wins:** Small architectures (3 hidden units) and online learning proved optimal for deterministic tasks (MONK 1/2), easily reaching 100% accuracy.
 - **Regularization Necessity:** For noisy data (MONK 3), unregularized models overfitted heavily. Introducing L2 regularization and Early Stopping improved test accuracy by $\sim 1.4\%$.
- **CUP Task (Regression):**
 - **Architecture:** Deeper and wider networks (2 layers, > 100 units) were required to capture the complex function manifold.
 - **Stability:** Full-batch learning and half batch learning combined with high momentum ($\alpha = 0.9$) provided the most stable convergence, while learning rate decay effectively mitigated oscillation in the final training stages.
 - **Final Comment:** we believe that due to the fixed epochs for the final training the resulting model could be fitting also the noise but we cannot be sure about that yet.

We agree to the disclosure and publication of my name, and of the results with preliminary and final ranking.

Grid searched parameters: since not all the runs were carried at the same time not every possible combination of these parameters have been explored, it manually became a nested grid search where we logged the results and performed a more specific grid search around the best values in later runs

Table 5: Combined Hyperparameter Search Space (CUP)

Category	Parameters	Values Explored
Architecture	Hidden Units (Leaky ReLU)	[800, 400], [600, 300], [500,250],[300, 150], [100, 50], [64, 32] , [100, 150, 150, 100], [150, 300, 150], [200, 100, 200], [400, 200, 100],
Optimization	Learning Rate (η) Momentum (α) Decay (Factor 0.95) Mini-Batch Size	0.001, 0.02, 0.005 0.0, 0.5, 0.9 Step: 25, 50, 100, 150, 200 epochs 32, 64, 50%, 100% (fullBatch)
Regularization	L2 Lambda (λ) Early Stopping Max Epochs	0, 10^{-6} Pat: 30, 50, 75, 100, 150, 200 (Eps: 10^{-6} , 10^{-7} , 10^{-8} , 10^{-9} , 10^{-10} , 10^{-11}) 400, 3000, 5000, 8000